

Wochenprotokoll: VisionQuest

Name: Matthias

Datum: 06.03.2026

Meilenstein: M5

Projektstatus:  Im Plan /  Verzögert /  Kritisch

1. Was wurde diese Woche gemacht?

- Camera-Plugin (camera) in Flutter integriert mit Permissions
- Live-Preview Screen mit CameraController implementiert
- Image Upload an Backend-Endpoint /api/detect mit MultipartRequest
- Erkennungsergebnisse als JSON zurück an Flutter
- Error-Handling für fehlende Bilder und Erkennungsfehler

2. Wie liegt man im Zeitplan?

- Im Zeitplan - Computer Vision Kernfunktionalität läuft
- Kamera funktioniert auf Android-Geräten stabil
- Backend verarbeitet Bilder asynchron mit Python subprocess

3. Was sind die nächsten Schritte?

- Flutter Frontend mit Login/Register Screens erweitern
- Alle API-Endpoints mit curl/Postman testen
- Token-Storage in Flutter implementieren
- Gamification-Features (XP, Levels, Streaks) testen
- npm Security Vulnerabilities beheben

4. KI-Einsatz & Dokumentation

Modell: GitHub Copilot (GPT-4)

1. Camera-Permission in Flutter

Prompt: "Wie implementiere ich Kamera-Permissions richtig in Flutter für Android und iOS?"

Ergebnis: Permission-Handler mit camera-Plugin, AndroidManifest.xml und Info.plist Konfiguration, Runtime-Permission-Request vor CameraController-Init

Integration: Permission-Checks werden beim Öffnen des Scanner-Screens durchgeführt. Bei fehlender Permission wird Dialog mit Erklärung angezeigt und User zu Settings weitergeleitet.

2. Live-Preview mit CameraController

Prompt: "CameraController zeigt nur schwarzen Screen, wie fixe ich das?"

Ergebnis: await controller.initialize() vor dem Build, FutureBuilder für async Loading, AspectRatio-Widget für korrekte Preview-Darstellung

Integration: CameraPreview wrapped in FutureBuilder mit Loading-Indicator. AspectRatio verhindert Verzerrungen auf verschiedenen Screen-Sizes.

3. Bild-Upload an Backend

Prompt: "Wie sende ich captured Image von Flutter an Node.js Backend?"

Ergebnis: XFile zu bytes konvertieren, MultipartRequest mit 'image' field, multer-Middleware im Backend für File-Parsing, temporäres Speichern in /uploads

Integration: Nach takePicture() wird XFile.readAsBytes() aufgerufen, dann MultipartRequest an /api/detect.

4. Python-Integration

Prompt: "Wie rufe ich Python Script aus Node.js auf und bekomme Ergebnisse zurück?"

Ergebnis: child_process.spawn mit Python-Path und Script-Args, stdout/stderr Event-Listener, JSON.parse für Output, Promise-Wrapper für async/await

Integration: Backend spawnst Python-Prozess mit Bildpfad als Argument. Python gibt JSON mit Detections aus, Node.js parsed das und sendet es an Flutter zurück.

5. Detection Error-Handling

Prompt: "Was passiert, wenn Model keine Objekte erkennt oder einen Fehler wirft?"

Ergebnis: Empty-Array bei 0 Detections, Try-Catch um Aufruf, Stderr-Logging für Python-Fehler, HTTP 500 mit aussagekräftiger Error-Message bei Crash

Integration: Frontend zeigt 'Keine Objekte erkannt'-Message bei leerem Array. Bei Server-Error wird Retry-Button mit Fehlerbeschreibung angezeigt.